



Institución
Universitaria
Reacreditada en Alta Calidad

ESTRUCTURA DE DATOS Y »»» LABORATORIO «««

580506004-8

William Giraldo Escobar
Facultad de Ingenierías
Departamento de Sistemas de Información



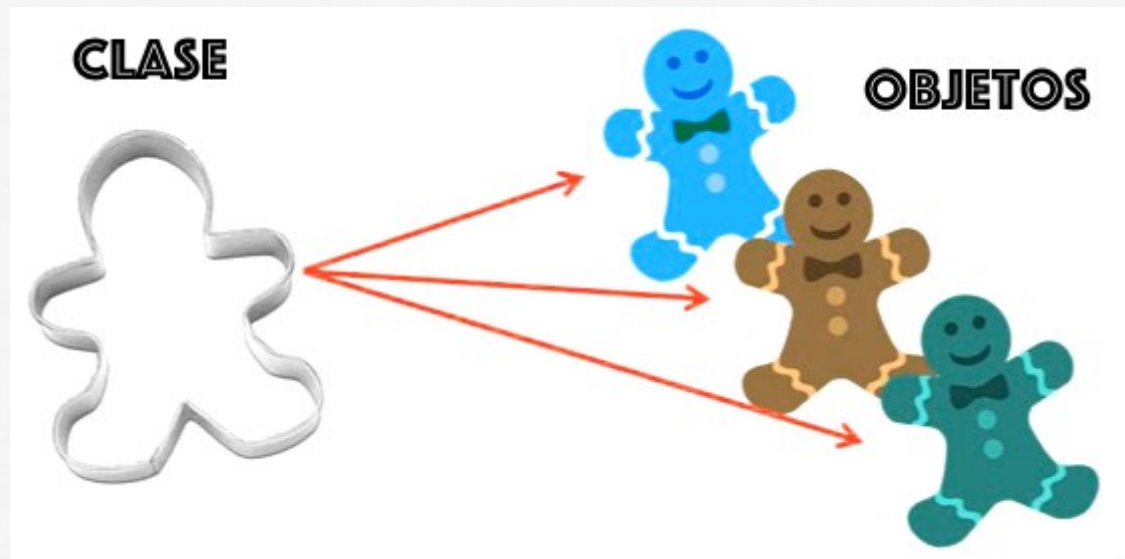
QUÉ ES PROGRAMACIÓN ORIENTADA A OBJETOS

Es una forma de programar en donde combinamos funciones y datos en un mismo objeto

- **CLASES:** es donde encapsulamos las funciones (métodos) y las características (atributos)
- **OBJETO:** es una instancia de esa clase

QUÉ ES PROGRAMACIÓN ORIENTADA A OBJETOS

- **CLASES:** es una plantilla donde se definen propiedades y comportamientos
- **OBJETO:** se crea a partir de esa plantilla pero se definen sus *valores* propios



CLASE JUGADOR

ATRIBUTOS (datos)

- Nombre del personaje
- Número de vidas
- Color de la ropa

MÉTODOS (funciones)

- Saltar
- Recolectar monedas
- Correr



Objeto 1 de la clase jugador

ATRIBUTOS (datos)

- Nombre del personaje= mario
- Número de vidas= 3
- Color de la ropa= rojo

MÉTODOS (funciones)

- Saltar
- Recolectar monedas
- Correr



Objeto 2 de la clase jugador

ATRIBUTOS (datos)

- Nombre del personaje= luigi
- Número de vidas= 3
- Color de la ropa= verde

MÉTODOS (funciones)

- Saltar
- Recolectar monedas
- Correr



CLASE ENEMIGO – objeto 1

ATRIBUTOS (datos)

- Nombre=bill
- Zona de ataque = aire
- Nivel de ataque = extremo

MÉTODOS (funciones)

- Atacar
- Avanzar hacia adelante



CLASE ENEMIGO – objeto 2

ATRIBUTOS (datos)

- Nombre = goomba
- Zona de ataque = tierra
- Nivel de ataque = medio

MÉTODOS (funciones)

- Atacar
- Avanzar hacia adelante



CLASE *JUGADOR*

Objeto 1



Objeto 2



ATRIBUTOS (datos)

Nombre del personaje, Número de vidas, Color de la ropa

MÉTODOS (funciones)

Saltar, Recolectar monedas, Correr

CLASE *ENEMIGO*

Objeto 1



Objeto 2



ATRIBUTOS (datos)

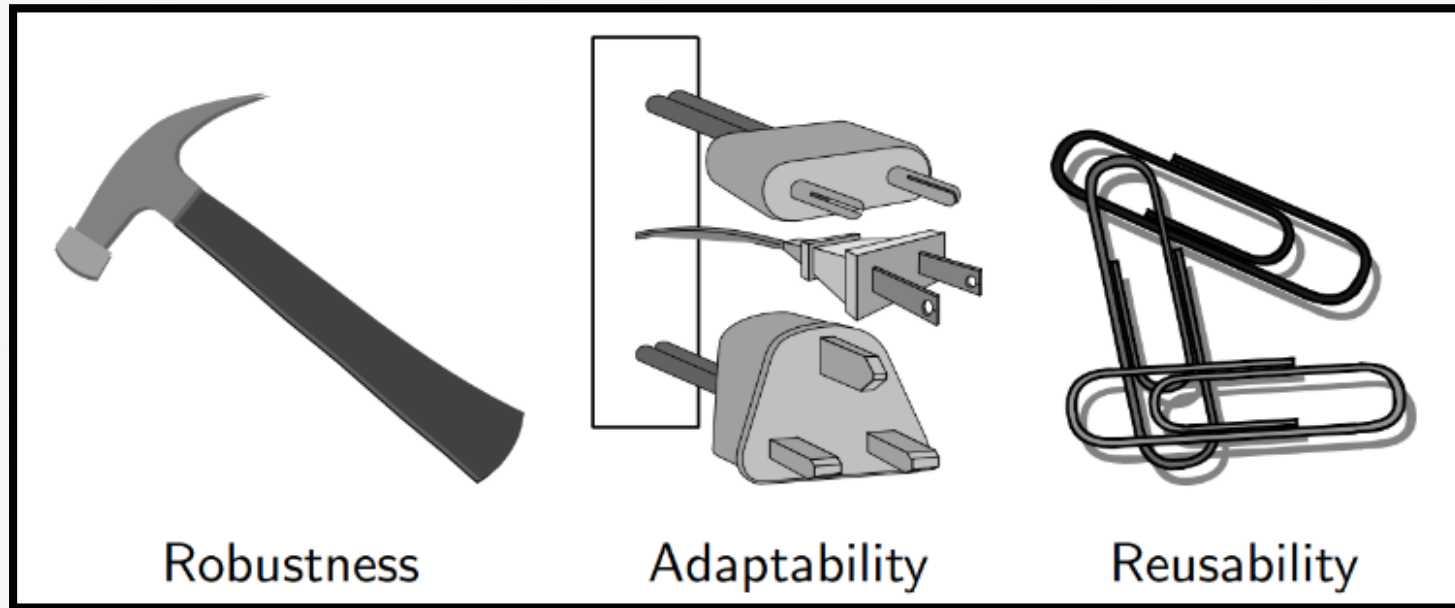
Nombre , Zona de ataque, Nivel de ataque

MÉTODOS (funciones)

Atacar, Avanzar hacia adelante

OBJETIVO DE LA POO

Crear algoritmos que permitan la **Robustez**, **Adaptabilidad** y **Reusabilidad**



ROBUSTEZ

- Es un programa que produce la salida correcta para todas las entradas posibles
- Puede manejar entradas inesperadas

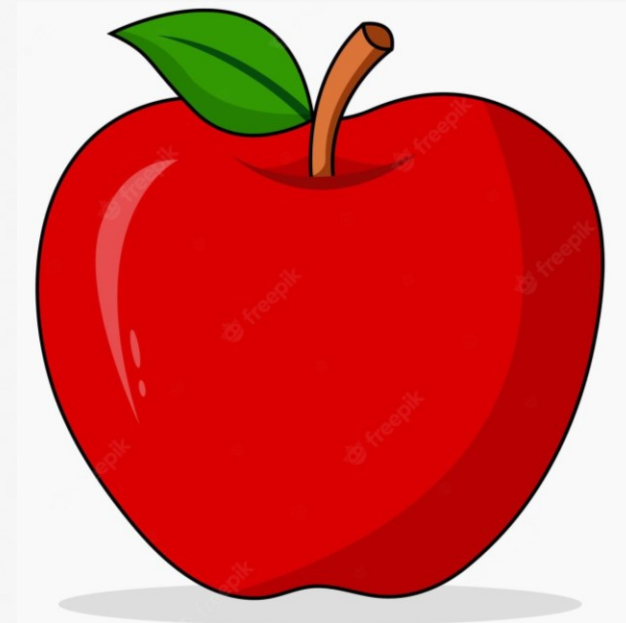


ROBUSTEZ - EJEMPLO

Un programa que le pide al usuario que ingrese el precio de una manzana

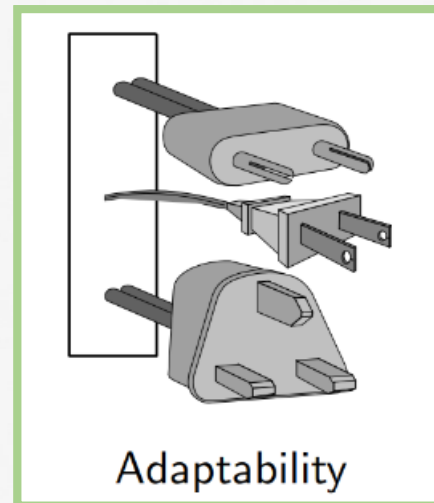
El usuario ingresa el valor negativo -2.000

Qué debería hacer el algoritmo?



ADAPTABILIDAD

- Que un programa pueda perdurar con el paso del tiempo
- Que se adapte al cambio
- Que se pueda implementar bajo diferentes condiciones



REUSABILIDAD

- Que un mismo programa se pueda utilizar en diferentes sistemas
- Que un mismo programa se pueda utilizar varias veces sin tener que reescribirlo
- La programación por funciones, así como la POO son claros ejemplos de reusabilidad



PRINCIPIOS DE LA POO

- **Modularidad:** dividir un programa en pequeños algoritmos (funciones)
- **Abstracción:** capacidad de representar un objeto de la vida real en un programa
- **Encapsulación:** agrupar los datos (atributos) y los métodos (funciones) dentro de una misma clase

DISEÑO ORIENTADO A OBJETOS

En el diseño se especifica la estructura de como un programa o sistema será escrito y funcionará, sin escribir la implementación completa.

El diseño es una transición de “que” debe hacer un sistema a “como” lo va a hacer, respondiendo las siguientes preguntas:

¿Qué clases se requieren?

¿Cuáles son los campos y métodos de cada clase?

¿Cómo cada clase interactúa con otras clases?



Institución
Universitaria
Reacreditada en Alta Calidad

PASOS PARA RESOLVER UN PROBLEMA DE POO

DESCRIPCIÓN DE REQUERIMIENTOS

Una biblioteca desea tener un sistema de información para administrar su colección bibliográfica.

Cada item de la biblioteca (ejemplo: libros, revistas, cds, etc.) tiene un identificador y pueden existir varios ejemplares de un mismo item.

El staff de la biblioteca puede prestar hasta 6 items por 15 días.

Un usuario registrado puede prestar hasta 3 items por 10 días.

Tanto del staff como de los usuarios registrados se almacena en el sistema su información de contacto (nombre, id, teléfono, email).

PASO 1

Identificar los *sustantivos* en la descripción.
Los sustantivos son los que conformaran las
clases o *atributos* de las clases.

PASO 1 – SUSTANTIVOS

Una **biblioteca** desea tener un **sistema de información** para administrar su **colección bibliográfica**.

Cada **ítem** de la **biblioteca** (ejemplo: **libros, revistas, cds**, etc.) tiene un **identificador** y pueden existir varios **ejemplares** de un mismo **ítem**.

El **staff** de la **biblioteca** puede prestar hasta 6 **ítems** por 15 días.

Un **usuario** registrado puede prestar hasta 3 **ítems** por 10 días.

Tanto del **staff** como de los **usuarios** registrados se almacena en el **sistema** su **información de contacto** (**nombre, id, teléfono, email**).



PASO 2

Identificar los *verbos*.

Los verbos son los que conformaran los *métodos* de las clases.

PASO 2 - VERBOS

Una biblioteca *desea tener* un sistema de información para *administrar* su colección bibliográfica.

Cada item de la biblioteca (ejemplo: libros, revistas, cds, etc.) *tiene* un identificador y *pueden existir* varios ejemplares de un mismo item.

El staff de la biblioteca *puede prestar* hasta 6 items por 15 días.

Un usuario registrado *puede prestar* hasta 3 items por 10 días.

Tanto del staff como de los usuarios registrados se *almacena* en el sistema su información de contacto (nombre, id, teléfono, email).



PASO 3

A partir de los sustantivos definir las clases y los atributos de las clases.



Sustantivos	Descripción y analisis
Biblioteca Sistema de información Sistema	representan el sistema completo a implementar
Colección bibliográfica	conjunto de items que hacen parte de la biblioteca. Esta colección puede representarse por un arreglo de items o por otra estructura de datos mas avanzada.
items	agrupa los diferentes tipos de objetos que pueden ser parte de la colección, como libros, revistas, cds, etc. Todos estos objetos dentro del sistema deben tener un identificador y el número de ejemplares
staff de la biblioteca usuarios registrados	tienen las mismas características, información de contacto, un numero de items que pueden prestar por determinado número de días, por tanto, se pueden representar por una misma clase

Los sustantivos los separamos en clases y atributos.

Sustantivo para clases	Sustantivos para atributos
• Biblioteca • Sistema de información • Sistema	• Colección bibliográfica
• Item (libro, revista, cds, etc.)	• Identificador • Cantidad de Ejemplares
• Staff de la biblioteca • Usuario registrado	• Información de contacto: nombre, id, teléfono, email • Numero de items que puede prestar • Número de días que puede prestar

Los sustantivos los separamos en clases y atributos.

Clase	Atributos
Biblioteca	• Colección
Item	• Id • Num_ejemplares
Usuario	• Nombre • Id • Tel • Email • Limite_items • Limite_dias



PASO 4

A partir de los verbos definir los métodos de las clases.

PASO 4

- En general, sistema debe permitir el préstamo y la devolución de un item.
- De forma similar, cada item debe poderse prestar y devolver, modificando el número de ejemplares disponibles
- Cada usuario debe poder prestar y devolver modificando el límite de items disponible para el usuario.

De todo lo anterior, el problema de POO se resume a programar:

Clase	Atributos	Métodos
Biblioteca	• Colección	• Prestar • Devolver
Item	• Id • Num_ejemplares	• Prestar • Devolver
Usuario	• Nombre • Id • Tel • Email • Limite_items • Limite_dias	• Prestar • Devolver

EL IDENTIFICADOR SELF

- Es una etiqueta que se utiliza para referirse al propio objeto dentro de sus métodos.
- Se usa para acceder a los atributos y métodos de una clase.

EL IDENTIFICADOR SELF

```
class MiClase:  
    def __init__(self, dato):  
        self.valor = valor  
  
    def mostrar_valor(self):  
        print(self.valor)
```

EL CONSTRUCTOR `__init__`

```
class MiClase:  
    def __init__(self, dato):  
        self.valor = valor  
  
    def mostrar_valor(self):  
        print(self.valor)
```

EL CONSTRUCTOR `__init__`

`__init__` se utiliza para configurar los valores iniciales de los atributos de un objeto cuando se crea una instancia de una clase. Se llama automáticamente.

Es como la receta que sigue el objeto para prepararse y tener todas sus características listas desde el principio.

Crear la clase

```
class MiClase:  
    def __init__(self, dato):  
        self.valor = dato  
  
    def mostrar_valor(self):  
        print(self.valor)
```

Crear un objeto de esa clase

```
# Crear una instancia de la clase  
objeto = MiClase(42)
```

Crear la clase

```
class MiClase:  
    def __init__(self, dato):  
        self.valor = dato  
  
    def mostrar_valor(self):  
        print(self.valor)
```

Acceder a los métodos

```
# Llamar al método en la instancia  
objeto.mostrar_valor() # Esto imprimirá "42"
```

Crear la clase

```
class MiClase:  
    def __init__(self, dato):  
        self.valor = dato  
  
    def mostrar_valor(self):  
        print(self.valor)
```

Acceder a las variables

```
# Llamar a la variable valor en la instancia  
objeto.valor
```

EJEMPLO

- Crear una clase LIBRO
- La clase LIBRO tiene un atributo que indica cuando un libro es digital o físico

(True = digital) (False=físico)

- Por defecto, todos los libros son digitales

SOLUCIÓN

```
class Book:  
    def __init__(self, valor=True):  
        self.is_electronic = valor
```

```
libro=Book()  
libro.is_electronic
```



EJEMPLO

- Añadir un método que retorne “El libro es Digital” o “El libro es Físico” según sea el caso

SOLUCIÓN

```
class Book:
    def __init__(self, valor=False):
        self.is_electronic = valor

    def tipo(self):

        if self.is_electronic == True:
            print("el libro es digital")
        else:
            print("el libro es fisico")
```

```
libro=Book(True)
libro.is_electronic
libro.tipo()
```



EJEMPLO

- Añadir un atributo “precio”
- Añadir un método que calcule el total a pagar dependiendo de la cantidad de libros que quiero llevar y del precio

SOLUCIÓN

```
class Book:
    def __init__(self,precio,valor=False):
        self.is_electronic = valor
        self.precio=precio

    def tipo(self):

        if self.is_electronic == True:
            print("el libro es digital")
        else:
            print("el libro es fisico")

    def total_a_pagar(self,cantidad):
        print("El total a pagar es ",self.precio*cantidad)
```

```
libro=Book(100)
libro.total_a_pagar(2)
libro.precio
```



DOCSTRING

El docstring en Python es una cadena de texto que se coloca al principio de una función, clase, método o módulo para proporcionar una descripción detallada de lo que hace ese elemento.

DOCSTRING

```
class Book():
```

```
    """
```

```
    Clase para trabajar con libros
```

```
    """
```

```
    is_electronic = False
```

DOCSTRING

```
class Book():
```

```
    """
```

```
    Clase para trabajar con libros
```

```
    """
```

```
    is_electronic = False
```

```
print(Book.__doc__)
```

DOCSTRING

- Para acceder a todos los atributos de un objeto y a los valores que tiene asignados, podemos usar el método `.__dict__`

```
book.__dict__
```




EL MÉTODO DESTRUCTOR

Así como existe un método constructor `__init__` , existe un método destructor cuyo cometido consiste en eliminar un objeto.

EL MÉTODO DESTRUCTOR

```
class Book:  
    def __init__(self,precio,valor=False):  
        self.is_electronic = valor  
        self.precio=precio  
  
    def __del__(self):  
        print("Acabas de llamar al método destructor. El objeto acaba de ser eliminado")
```

del book

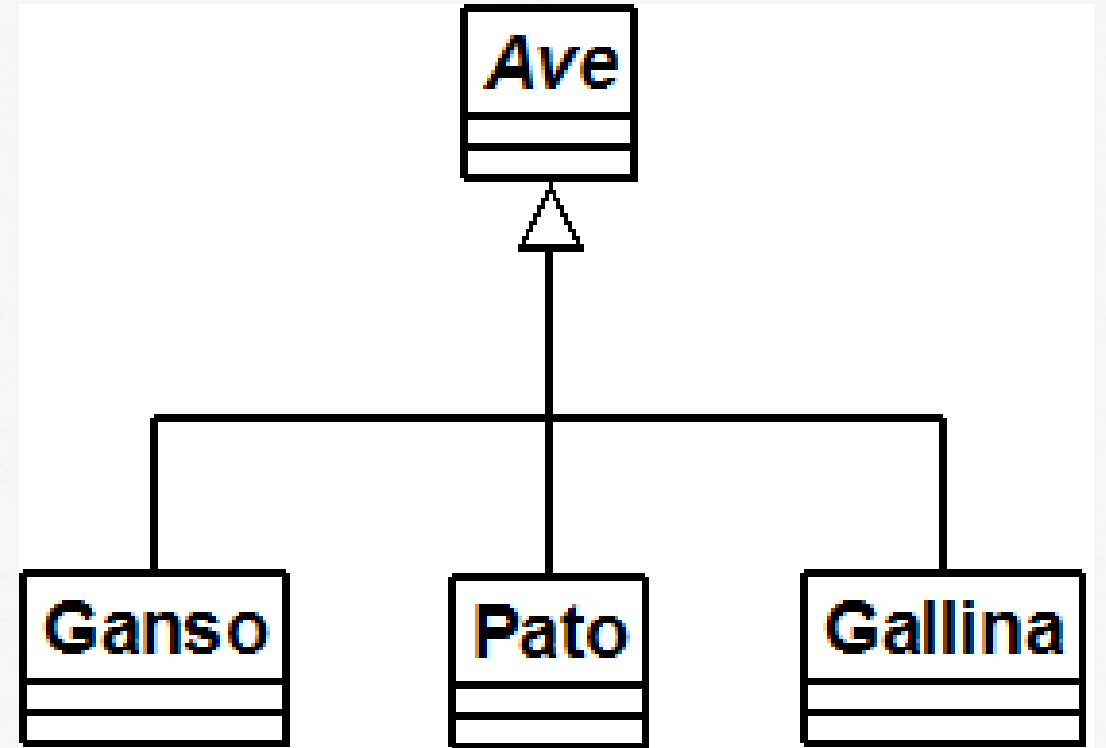
HERENCIA

La herencia es un concepto fundamental en la programación orientada a objetos (POO) que permite crear *nuevas clases* basadas en clases ya existentes.

Permite crear nuevas clases que heredan los *mismos atributos* y *métodos* de una clase creada anteriormente.

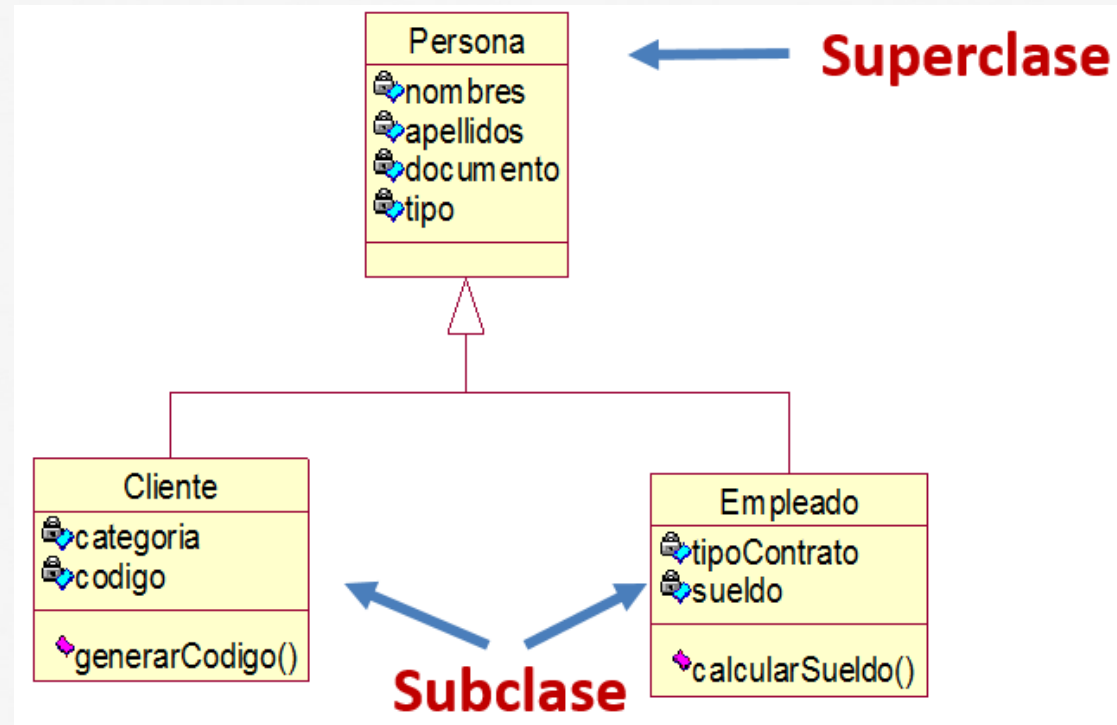
HERENCIA

Qué tienen en común las clases Ganso, Pato y Gallina con la clase Ave ?



HERENCIA

Es útil cuando ya existe una clase que hace todo lo que necesitamos.

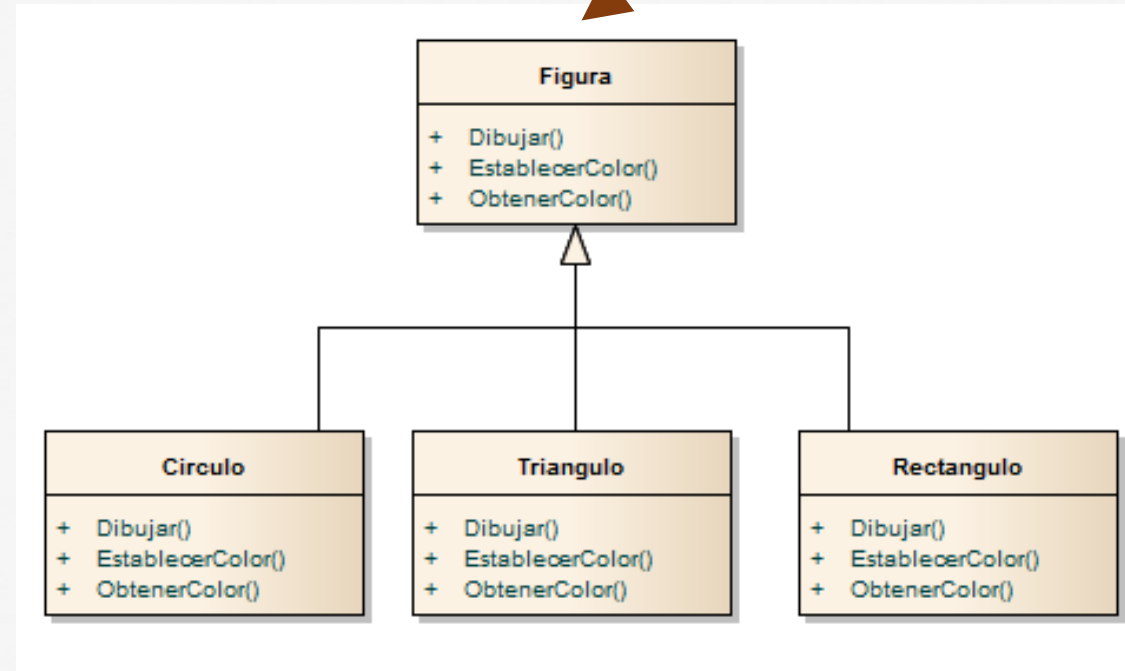


HERENCIA

- La herencia consiste en definir una nueva clase a partir de una existente.
- La clase existente se *denomina clase base, padre o super clase*.
- La nueva clase se denomina *subclase o clase hija*.

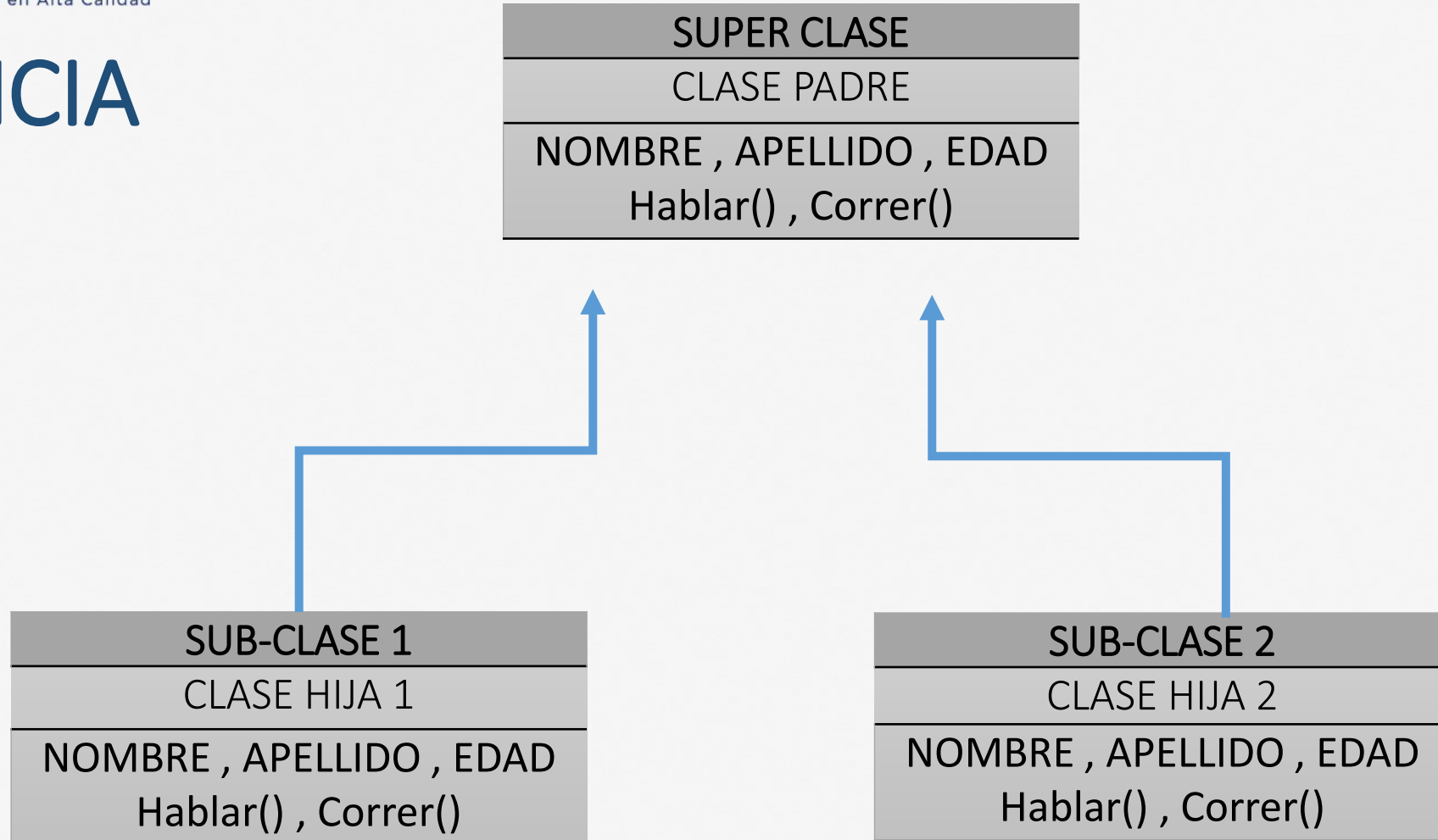
HERENCIA

CLASE PADRE O SUPERCLASE



CLASES HIJAS O SUB-CLASES

HERENCIA



HERENCIA

- La clase hija hereda automáticamente **todos** los métodos y los atributos de la clase padre, **excepto** los *constructores* (constructor `__init__`).
- A la subclase se le pueden añadir *nuevos métodos* y *nuevos atributos* que no estén contemplados en la clase padre.

¿Qué diferencia hay entre la clase Bebe y la clase Adulto?

```
class bebe():
```

```
    es_adulto = False
```

```
    def __init__(self, nombre, apellido, edad):
```

```
        self.nombre = nombre
```

```
        self.apellido = apellido
```

```
        self.edad = edad
```

```
    def nombre_completo(self):
```

```
        return "{}{}".format(self.nombre, self.apellido)
```

```
class adulto():
```

```
    es_adulto = True
```

```
    def __init__(self, nombre, apellido, edad):
```

```
        self.nombre = nombre
```

```
        self.apellido = apellido
```

```
        self.edad = edad
```

```
    def nombre_completo(self):
```

```
        return "{}{}".format(self.nombre, self.apellido)
```

¿Cuáles son los métodos y los atributos en común?

```
class bebe():
```

```
    es_adulto = False
```

```
    def __init__(self, nombre, apellido, edad):
```

```
        self.nombre = nombre
```

```
        self.apellido = apellido
```

```
        self.edad = edad
```

```
    def nombre_completo(self):
```

```
        return "{}{}".format(self.nombre, self.apellido)
```

```
class adulto():
```

```
    es_adulto = True
```

```
    def __init__(self, nombre, apellido, edad):
```

```
        self.nombre = nombre
```

```
        self.apellido = apellido
```

```
        self.edad = edad
```

```
    def nombre_completo(self):
```

```
        return "{}{}".format(self.nombre, self.apellido)
```

HERENCIA

- Del ejemplo anterior podemos ver que los atributos entre la clase bebe y la clase adulto son los mismos.
- La clase bebe y la clase adulto tienen nombre , apellido, edad, y un método que imprime el nombre completo.

Con la Herencia , nos podemos ahorrar el trabajo de definir dos veces los atributos y métodos que tienen en común.



HERENCIA

- La herencia nos permite “copiar” los métodos y los atributos de una clase a otra.
- Nos permite “reciclar” Código.
- Nos ahorra tiempo de trabajo.
- Si una clase ya tiene todo lo que necesito, puedo heredar de ella.

DEL EJEMPLO ANTERIOR

- la única diferencia entre ambas clases es el atributo `.es_adulto` que en `bebe` vale *False* y en `adulto` vale *True*

```
class bebe():  
  
    es_adulto = False  
  
    ...
```

```
class adulto():  
  
    es_adulto = True  
  
    ...
```


EJEMPLO BÁSICO DE HERENCIA

Creamos una clase Persona y apartir de ahí creamos las sub-clases bebe y adulto

```
class persona():  
  
    def __init__(self, nombre, apellido, edad):  
        self.nombre = nombre  
        self.apellido = apellido  
        self.edad = edad  
  
    def nombre_completo(self):  
        return "{}{}".format(self.nombre, self.apellido)
```

```
class bebe(persona):  
    es_adulto = False
```

```
class adulto(persona):  
    es_adulto = True
```

EJEMPLO BÁSICO DE HERENCIA

1. Cree un objeto bebe llamado ana cardona de 2 años de edad.
2. En una celda aparte invoque el nombre del bebe (recuerde usar el .nombre)
3. Invoque también la edad.
4. Cree un objeto adulto llamado Esteban Paniagua de 35 años de edad.
5. En una celda aparte invoque la edad.
6. Invoque también el apellido.

EJEMPLO DE HERENCIA

```
Bebe1=bebe('ana','cardona',2)
```

```
Bebe1.nombre
```

```
Bebe1.edad
```

```
adulto1=adulto('esteban',paniagua',35)
```

```
adulto1.edad
```

```
adulto1.apellido
```

EJEMPLO - HERENCIA

Crea una jerarquía de clases que representen diferentes tipos de vehículos. Tendrás una clase **padre** llamada Vehiculo, y dos clases **hijas** llamadas Automovil y Motocicleta.

Primero: define la clase **Vehiculo** con los siguientes atributos y métodos:

- *Atributos:* marca, modelo
- Método `__init__` para inicializar la marca y el modelo.
- Método *describir* que imprima información sobre el vehículo, como "Este es un vehículo de marca [marca] y modelo [modelo]."

EJEMPLO - HERENCIA

Primero: define la clase **Vehiculo**

```
class Vehiculo:
    def __init__(self, marca, modelo):
        self.marca = marca
        self.modelo = modelo

    def describir(self):
        print(f"Este es un vehículo de marca {self.marca} y modelo {self.modelo}.")
```

EJEMPLO - HERENCIA

Segundo: Define la clase **Automovil** que *herede* de Vehiculo con los siguientes atributos y métodos:

- *Atributo* adicional: puertas
- Método `__init__` que también reciba la cantidad de puertas.
- *El método* `describir_auto` para incluir la cantidad de puertas en la descripción.

EJEMPLO - HERENCIA

Segundo: Define la clase **Automovil** que *herede* de Vehiculo

```
class Automovil(Vehiculo):  
    def __init__(self, marca, modelo, puertas):  
        Vehiculo.__init__(self, marca, modelo) # Llamada al constructor de la clase base  
        self.puertas = puertas  
  
    def describir_auto(self): # Sobrescritura del método describir  
        print(f"Este es un automóvil de marca {self.marca}, modelo {self.modelo} y {self.puertas} puertas.")
```


EJEMPLO - HERENCIA

NOTA: RECUERDA QUE EL `__init__` NO SE HEREDA, POR ESO DEBEMOS LLAMARLO DE LA SIGUIENTE FORMA

```
class Automovil(Vehiculo):  
    def __init__(self, marca, modelo, puertas):  
        Vehiculo.__init__(self, marca, modelo)  
        self.puertas = puertas
```

EJEMPLO - HERENCIA

Tercero: Define la clase *Motocicleta* que herede de *Vehiculo* con los siguientes atributos y métodos:

- *Atributo adicional:* cilindrada
- Método `__init__` que también reciba la cilindrada.
- El *método* `describir_moto` para incluir la cilindrada en la descripción.

EJEMPLO - HERENCIA

Tercero: Define la clase *Motocicleta* que herede de *Vehiculo*

```
class Motocicleta(Vehiculo):  
    def __init__(self, marca, modelo, cilindrada):  
        Vehiculo.__init__(self, marca, modelo) # Llamada al constructor de la clase base  
        self.cilindrada = cilindrada  
  
    def describir_moto(self): # Sobrescritura del método describir  
        print(f"Esta es una motocicleta de marca {self.marca}, modelo {self.modelo} y cilindrada {self.cilindrada}.")
```

EJEMPLO - HERENCIA

Cuarto: Crea *objetos* de las clases *Automovil* y *Motocicleta*, y llama al método *describir* para cada una de ellas. Adicional a eso llama el método *describir_auto* y *describir_moto*

EJEMPLO - HERENCIA

Cuarto: Crea *objetos* de las clases *Automovil* y *Motocicleta*, y llama al método *describir* para cada una de ellas.

```
# Crear instancias y llamar al método describir
automovil = Automovil("Toyota", "Corolla", 4)
motocicleta = Motocicleta("Harley-Davidson", "Sportster", "1200cc")

automovil.describir()
motocicleta.describir()

automóvil.describit_auto()
motocicleta.describir_moto()
```

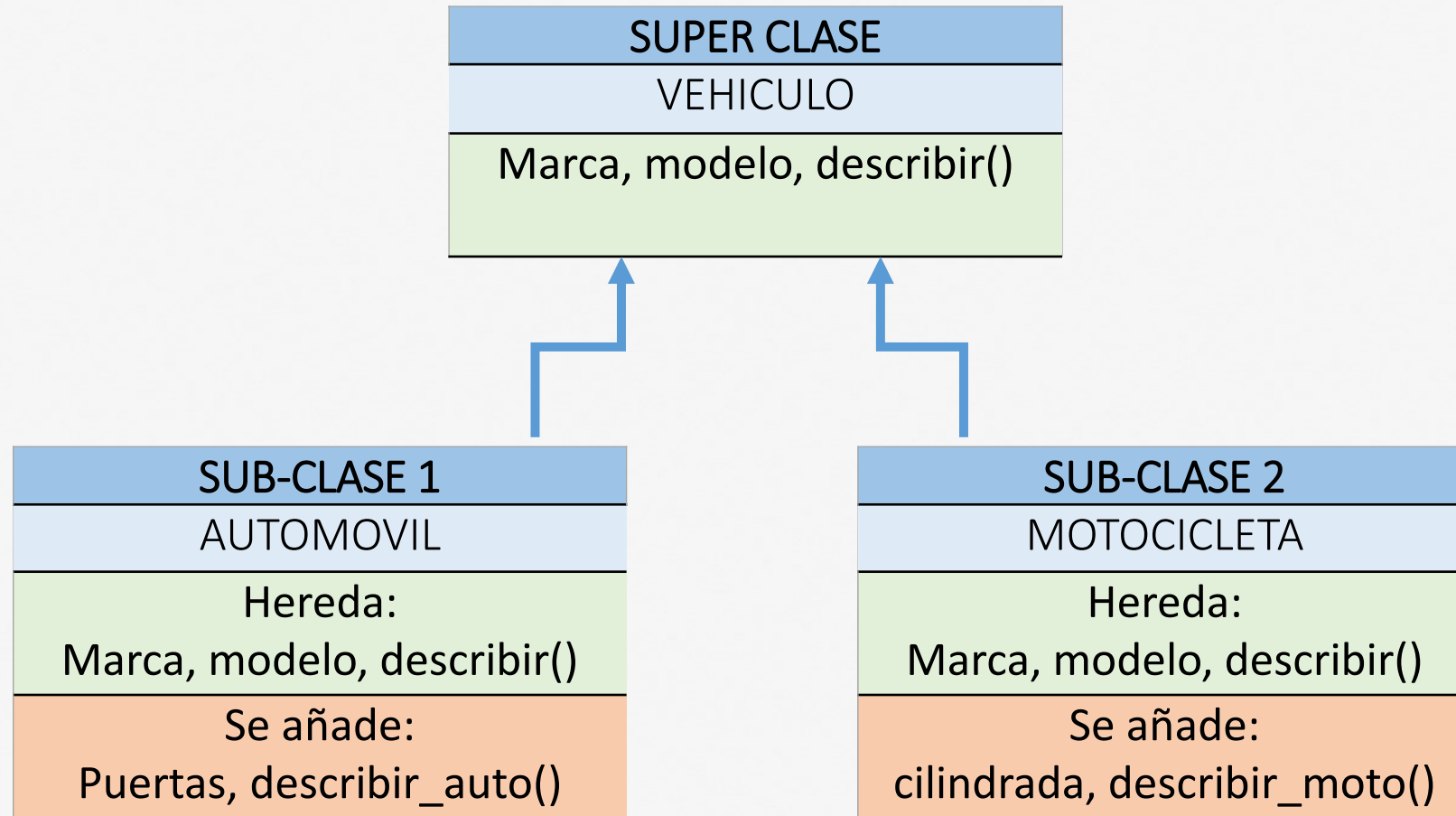
EJEMPLO - HERENCIA

IMPORTANTE: Note que los métodos *describir*, *describir_auto* y *describir_moto* son *tres métodos diferentes*.

El método *describir* se *hereda* por la clase automóvil y la clase motocicleta. Es decir, esas dos clases acceden al *mismo método*. Se comparte entre ambas.

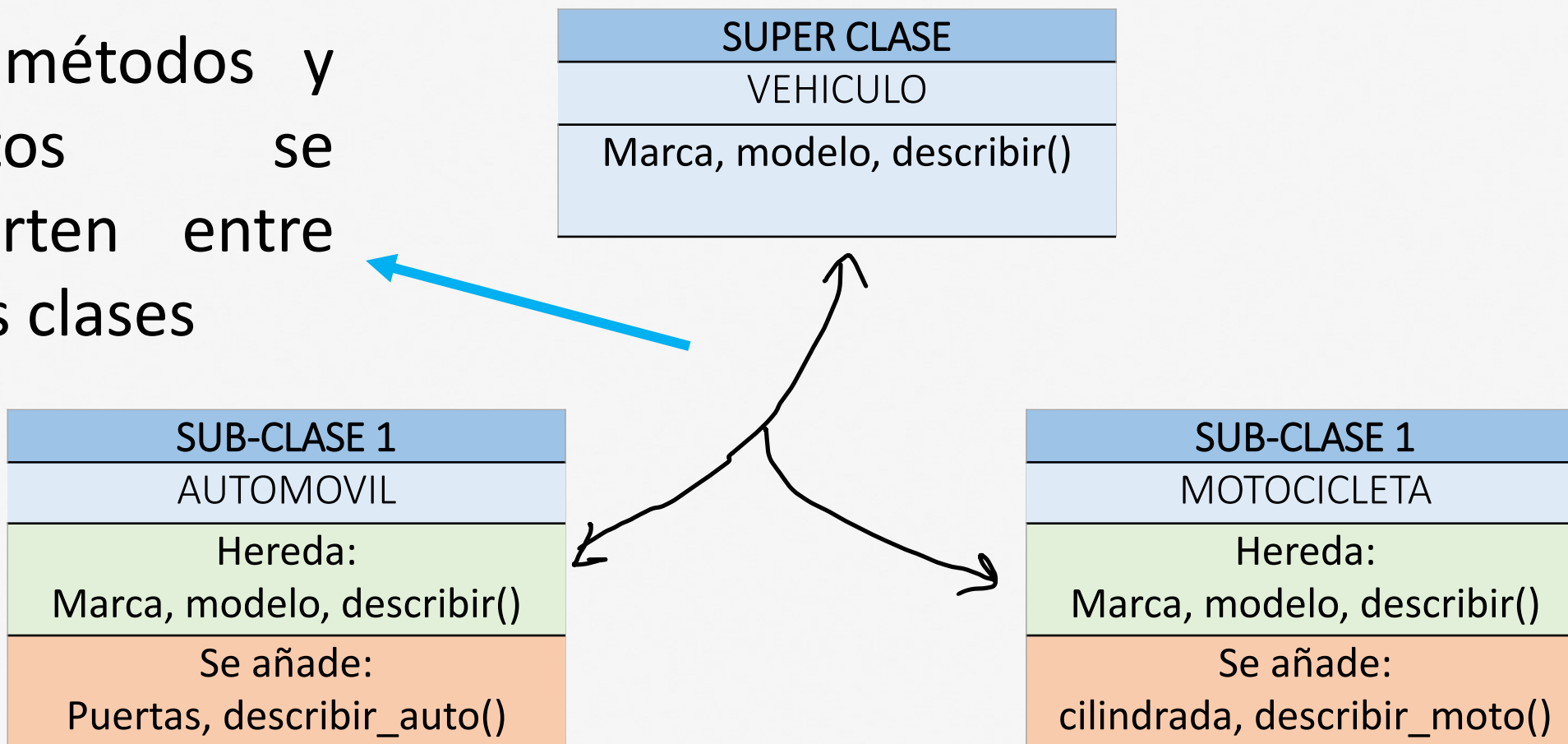
Los métodos *describir_auto* y *describir_moto* son *independientes* y son *únicos* para el automóvil y para la motocicleta respectivamente. No se comparten entre sí.

EJEMPLO – REPRESENTACIÓN GRÁFICA



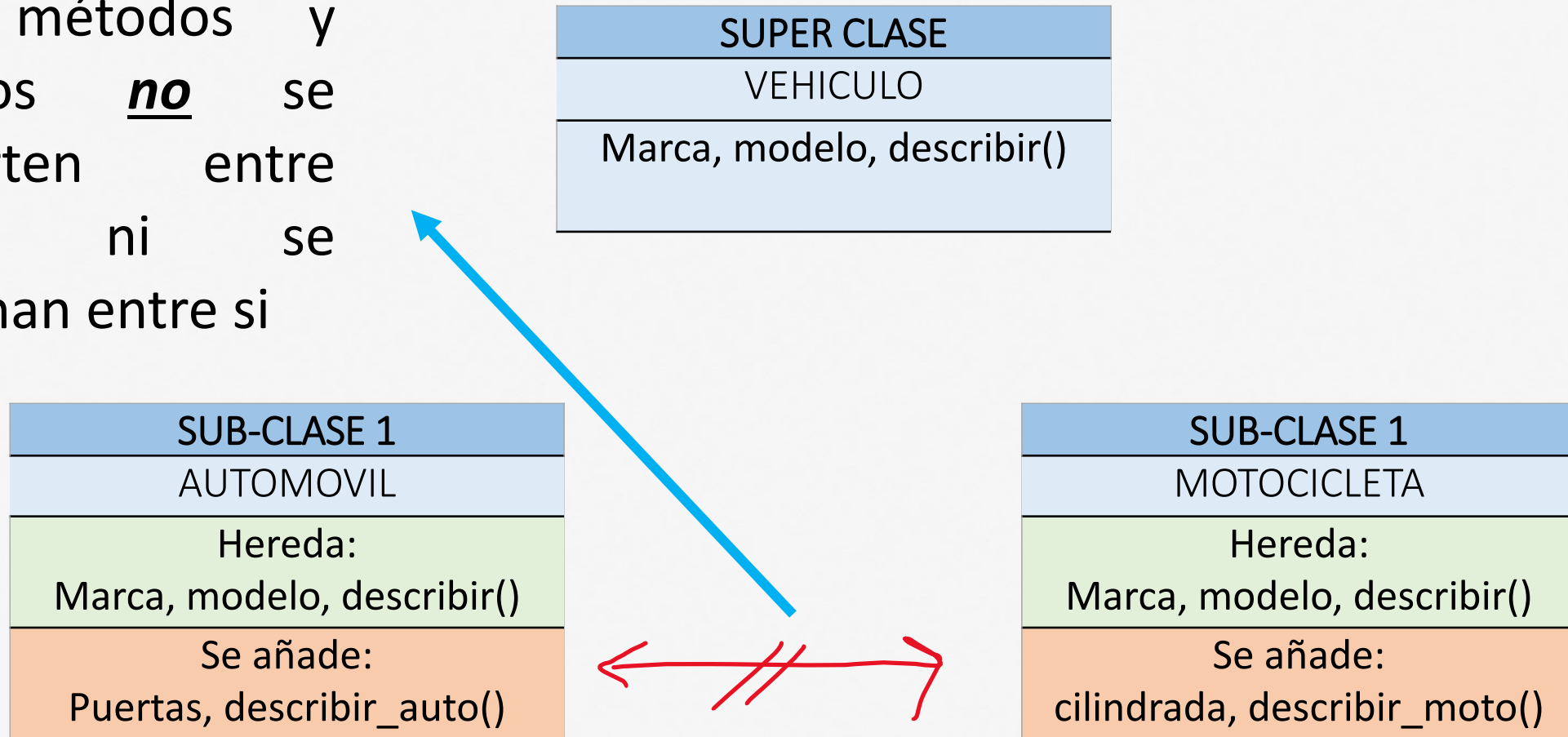
EJEMPLO – REPRESENTACIÓN GRÁFICA

Estos métodos y atributos se comparten entre las tres clases



EJEMPLO – REPRESENTACIÓN GRÁFICA

Estos métodos y atributos **no** se comparten entre clases, ni se relacionan entre si





Institución
Universitaria
Reacreditada en Alta Calidad

¡MUCHAS GRACIAS!

A decorative graphic element consisting of several horizontal lines in various colors (yellow, orange, red, purple, blue, green) and a stylized white swoosh or arc on the right side, positioned below the main text.

..... Hacia una era de
Universidad y *Humanidad*